



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Experiment: 06

Student Name: Priyanshu Choudhary

Branch: BE CSE

Semester: 6th

Subject Name: Full Stack II

UID: 23BCS12648

Section/Group: KRG_3B

Date of Performance: 05/02/26

Subject Code: 23CSH-309

1. Aim

To design and implement a Spring Boot REST API using Layered Architecture, applying Inversion of Control (IoC) and Dependency Injection (DI) to build a structured backend for the HealthHub application, including CRUD operations, DTO mapping, validation, and global exception handling.

2. Objectives

After completing this experiment, the following objectives were achieved:

- Understand Spring Boot fundamentals: Learned the role of Spring Boot in backend development by configuring and running a basic application.
- Apply IoC and Dependency Injection: Understood how Spring manages object creation and dependencies using annotations like `@Autowired`, `@Service`, and `@Repository`.
- Implement Layered Architecture: Structured the application using Controller, Service, and Repository layers to understand the separation of concerns in backend design.
- Develop RESTful APIs: Created CRUD operations using Spring Boot REST controllers and proper HTTP methods (GET, POST, PUT, DELETE).
- Use DTOs for API communication: Implemented DTO mapping to separate internal database models from API responses.
- Implement Validation: Applied Jakarta Validation annotations (`@NotNull`, `@Email`, etc.) to secure incoming data.

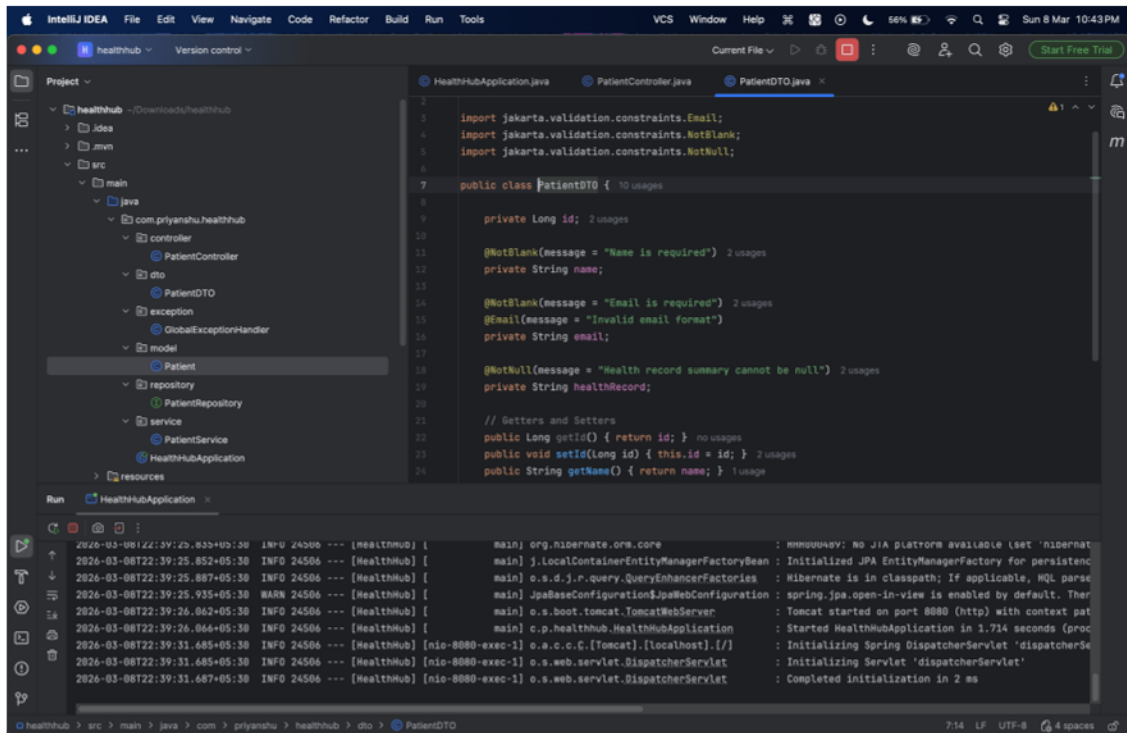
- Handle Exceptions Globally: Implemented centralized exception handling using `@ControllerAdvice`.
- Build the Health Hub Backend: Developed the functional backend services for patient and health record management.

3. Procedure

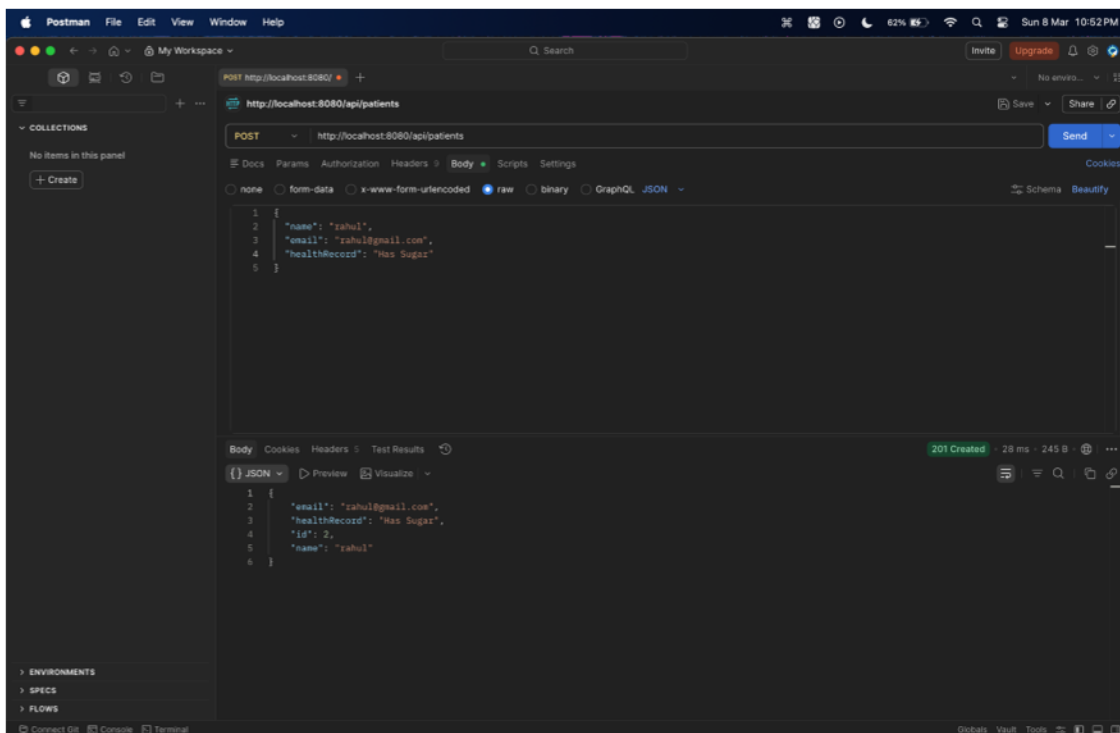
1. Project Initialization: Generated a Spring Boot project via Spring Initializr, including dependencies for Spring Web, Spring Data JPA, Validation, and H2 Database.
 2. Package Organization: Established a Layered Architecture by creating distinct packages: `model`, `dto`, `repository`, `service`, `controller`, and `exception`.
 3. Entity and DTO Creation: * Created a `Patient` entity mapped to a database table using JPA annotations (`@Entity`, `@Id`, `@GeneratedValue`).
 - Created a `PatientDTO` class utilizing Jakarta Validation annotations (`@NotBlank`, `@Email`) to ensure data integrity during API communication.
 4. Repository Layer Implementation: Created a `PatientRepository` interface extending `JpaRepository` to inherit built-in database CRUD operations without writing raw SQL.
 5. Service Layer Implementation: Developed a `PatientService` class annotated with `@Service`. Applied Dependency Injection via `@Autowired` to inject the repository and wrote the business logic for creating and retrieving patients.
 6. Controller Layer Implementation: Built a `PatientController` annotated with `@RestController` and mapped to `/api/patients`. Implemented `@PostMapping` and `@GetMapping` methods, utilizing the `@Valid` annotation to enforce DTO constraints on incoming requests.
 7. Global Exception Handling: Created a `GlobalExceptionHandler` annotated with `@ControllerAdvice` to intercept `MethodArgumentNotValidException` errors, ensuring that validation failures return structured `400 Bad Request` JSON responses instead of server stack traces.
 8. Testing: Executed the application and verified the API endpoints using Postman, testing both valid data submission and validation failure scenarios.
-

4. Screenshot Attachments

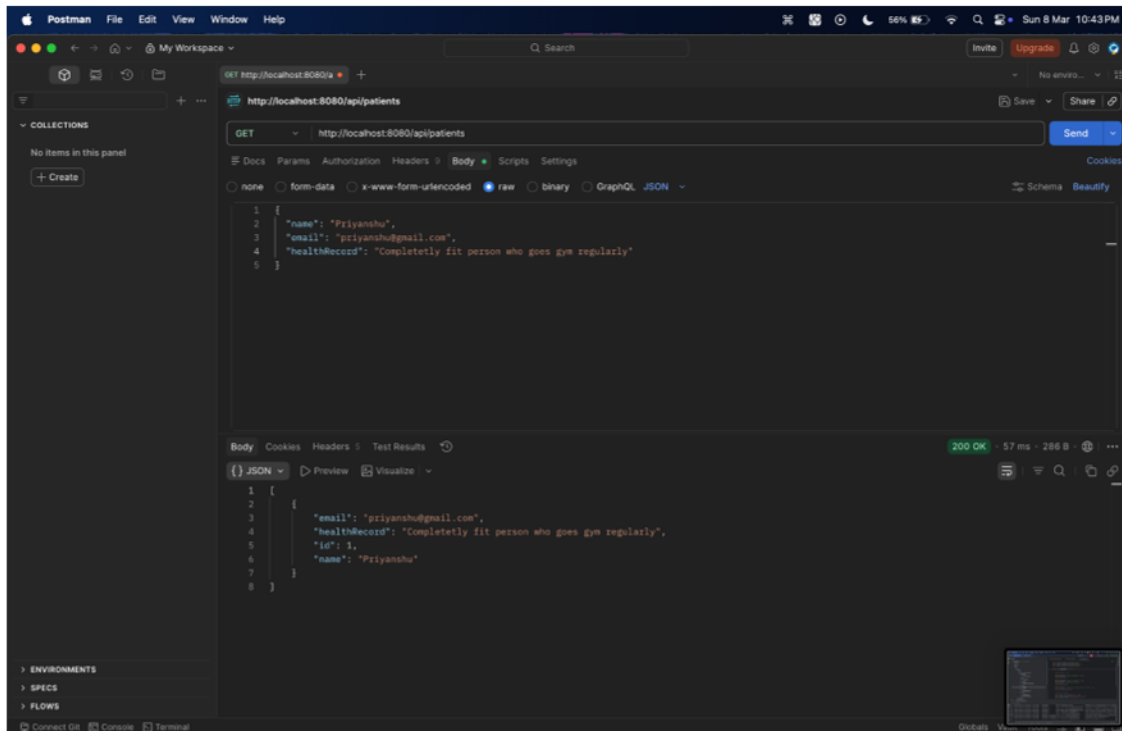
4.1. Code Implementation



4.2. Postman Testing – POST Request (Creating a Patient)



4.3. Postman Testing – GET Request (Retrieving All Patients)



5. Summary

This experiment involved designing and implementing a Spring Boot REST API using Layered Architecture to build a structured backend for the HealthHub application. The application was divided into Controller, Service, and Repository layers to ensure a clean separation of concerns. Key functionalities developed included RESTful CRUD operations using proper HTTP methods, DTO mapping to separate internal database models from API responses, and Jakarta Validation to secure incoming client data. Additionally, Inversion of Control (IoC) and Dependency Injection (DI) were applied throughout the layers, and global exception handling was integrated using `@ControllerAdvice` to manage errors effectively.

6. Result

The HealthHub backend module for patient and health record management was successfully developed and configured. The Spring Boot application effectively manages object creation and dependencies. Testing via Postman confirmed that the RESTful APIs accurately process HTTP POST and GET requests for patient data, enforce DTO validation constraints, and return the correct JSON responses and HTTP status codes.